

SEARCHING

Searching Definition:

Searching in data structures refers to the process of locating a specific item or element within a collection of data. The goal of searching is to determine whether the item is present in the data structure and, if so, to identify its location or position within the structure.

There are various searching algorithms and techniques used in data structures, depending on the type and characteristics of the data structure, the size of the collection, and the expected frequency and distribution of the target item.

Some common searching algorithms include

- linear search
- binary search
- hash-based search
- tree-based search

Linear search involves examining each element in the data structure one by one until the target item is found, making it a simple but potentially slow algorithm for large collections.

Binary search is a more efficient algorithm that requires the data to be sorted and involves dividing the collection in half at each step to quickly eliminate half of the remaining elements until the target item is found.

Sequential search (Linear Search)

Sequential search, also known as linear search, is a simple algorithm used to find a target element in a list or array. It works by sequentially checking each element of the list until the target element is found or the entire list is exhausted.

The algorithm begins by comparing the target element with the first element of the list. If they match, the search is successful and the algorithm returns the index of the element. If they do not match, the algorithm moves to the next element of the list and repeats the process until the target element is found or the end of the list is reached.

Sequential search is easy to implement and works for both ordered and unordered lists, but it can be relatively slow for large lists, as it has a worst-case time complexity of $O(n)$, where n is the number of elements in the list. Therefore, for larger lists, binary search algorithm may be more efficient.

Sequential search is useful in situations where the list is small, the target element is near the beginning of the list, or the list is unordered. It is also useful as a simple baseline algorithm for comparing the performance of other search algorithms.

Let us go through an example.

12	11	25	18	45	48	10
-----------	-----------	-----------	-----------	-----------	-----------	-----------

Assume the element 45 is to be searched from a sequence of elements given in the above list.

12	11	25	18	45	48	10
-----------	-----------	-----------	-----------	-----------	-----------	-----------

The Sequential search starts from the first element 12 (Index=0), here 12 is not equal to 45. So, it jumps to the next element 11.

12	11	25	18	45	48	10
-----------	-----------	-----------	-----------	-----------	-----------	-----------

So, 11 (index=1) will be compared with 45, and they are not equal. So, it jumps to the next element.

12	11	25	18	45	48	10
-----------	-----------	-----------	-----------	-----------	-----------	-----------

45 will be compared with 25 (Index=2), and they are not equal. So, it jumps to the next element.

12	11	25	18	45	48	10
-----------	-----------	-----------	-----------	-----------	-----------	-----------

45 will be compared with 18 (Index=3), and they are not equal. So, it jumps to the next element.

12	11	25	18	45	48	10
-----------	-----------	-----------	-----------	-----------	-----------	-----------

Now, 45 will be compared with 45 (Index=4), and they are equal. So, it returns the number is found in the position 4. Once the element to be searched is found, it will not go on searching for the same in the remaining elements.

Sequential Search Algorithm:

For each element in the array arr, repeat steps 2-3:

1. Initialize the index i to the first element in the array.
2. Compare the element at index i with the target element:
 - If the element at arr[i] is equal to the target element, return i.
 - If the element at arr[i] is not equal to the target element, go to step 3.
3. If the target element is not found in the array, return -1.

```
function sequential_search (list, target):  
    for i in range(Len(list)):  
        if list[i] == target:  
            return i  
    return -1
```

Program

```
#include<stdio.h>  
#include<conio.h>  
void main()  
{  
    int a[50],x,i,n,found=0;  
    printf("Enter limit For SEQUENTIAL SEARCH\n");  
    scanf("%d",&n);  
    printf("Enter %d elements\n",n);  
    for(i=0;i<n;i++)  
    {  
        scanf("%d",&a[i]);  
    }  
    printf("Enter a number to be search\n");  
    scanf("%d",&x);  
    for(i=0;i<n;i++)  
    {  
        if(a[i]==x)  
        {  
            found=1;  
            break;  
        }  
    }  
}
```

```
    if(found!=0)
        printf("%d is found in the list, at position %d\n",x,i);
    else
        printf("%d is not in the list\n",x);
    getch();
}
```

Selection Sort

In selection sort, the smallest value among the unsorted elements of the array is selected in every pass and inserted to its appropriate position into the array. It is also the simplest algorithm. It is an in-place comparison sorting algorithm.

In selection sort, the first smallest element is selected from the unsorted array and placed at the first position. After that second smallest element is selected and placed in the second position. The process continues until the array is entirely sorted.

Selection sort is generally used when -

- A small array is to be sorted
- Swapping cost doesn't matter
- It is compulsory to check all elements

Working of Selection sort Algorithm

To understand the working of the Selection sort algorithm, let's take an unsorted array. It will be easier to understand the Selection sort via an example.

12	29	25	8	32	17	40
----	----	----	---	----	----	----

Now, for the first position in the sorted array, the entire array is to be scanned sequentially.

At present, **12** is stored at the first position, after searching the entire array, it is found that **8** is the smallest value.

12	29	25	8	32	17	40
----	----	----	---	----	----	----

So, swap 12 with 8. After the first iteration, 8 will appear at the first position in the sorted array.

8	29	25	12	32	17	40
---	----	----	----	----	----	----

For the second position, where 29 is stored presently, we again sequentially scan the rest of the items of unsorted array. After scanning, we find that 12 is the second lowest element in the array that should be appeared at second position.

8	29	25	12	32	17	40
---	----	----	----	----	----	----

Now, swap 29 with 12. After the second iteration, 12 will appear at the second position in the sorted array. So, after two iterations, the two smallest values are placed at the beginning in a sorted way.

8	12	25	29	32	17	40
---	----	----	----	----	----	----

The same process is applied to the rest of the array elements. Now, we are showing a pictorial representation of the entire sorting process.

8	12	25	29	32	17	40
---	----	----	----	----	----	----

8	12	25	29	32	17	40
---	----	----	----	----	----	----

8	12	17	29	32	25	40
---	----	----	----	----	----	----

8	12	17	29	32	25	40
---	----	----	----	----	----	----

8	12	17	29	32	25	40
---	----	----	----	----	----	----

8	12	17	25	32	29	40
---	----	----	----	----	----	----

8	12	17	25	32	29	40
---	----	----	----	----	----	----

8	12	17	25	32	29	40
---	----	----	----	----	----	----

8	12	17	25	29	32	40
---	----	----	----	----	----	----

Now, the array is completely sorted.

Quick Sort

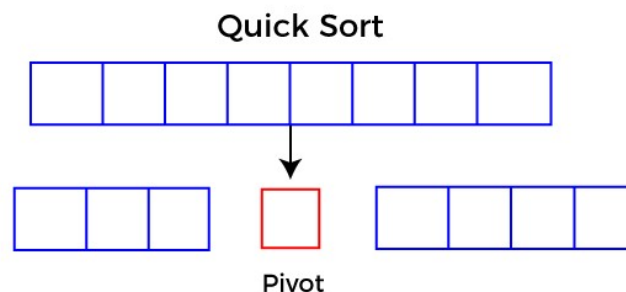
Quick sort is the widely used sorting algorithm that makes comparisons in average case for sorting an array of n elements. It is a faster and highly efficient sorting algorithm. This algorithm follows the divide and conquers approach.

Divide and conquer is a technique of breaking down the algorithms into sub problems, then solving the sub problems, and combining the results back together to solve the original problem.

- **Divide:** In Divide, first pick a pivot element. After that, partition or rearrange the array into two sub-arrays such that each element in the left sub-array is less than or equal to the pivot element and each element in the right sub-array is larger than the pivot element.
- **Conquer:** Recursively, sort two subarrays with Quick sort.
- **Combine:** Combine the already sorted array.

Quick sort picks an element as pivot, and then it partitions the given array around the picked pivot element. In quick sort, a large array is divided into two arrays in which one holds values that are smaller than the specified value (Pivot), and another array holds the values that are greater than the pivot.

After that, left and right sub-arrays are also partitioned using the same approach. It will continue until the single element remains in the sub-array.



Choosing the pivot:

Some of the ways of choosing a pivot are as follows -

- Pivot can be random, i.e. select the random pivot from the given array.
- Pivot can either be the rightmost element of the leftmost element of the given array.
- Select median as the pivot element.

Algorithm:

```
QUICKSORT (array A, start, end)
{
    if (start < end)
    {
        p = partition(A, start, end)
        QUICKSORT (A, start, p - 1)
        QUICKSORT (A, p + 1, end)
    }
}
```

Working of Quick Sort Algorithm

To understand the working of quick sort, let's take an unsorted array. It will make the concept more clear and understandable.

24	9	29	14	19	27
----	---	----	----	----	----

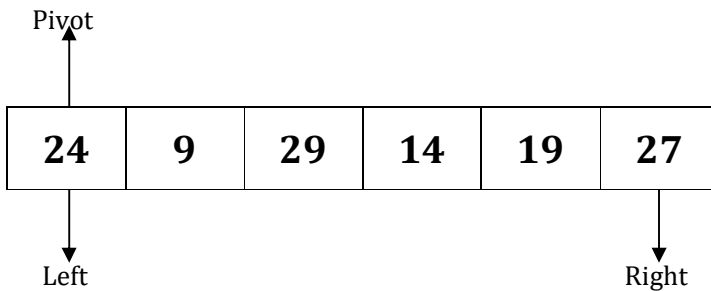
In the given array, we consider the leftmost element as pivot. So, in this case,

$a[\text{left}] = 24$

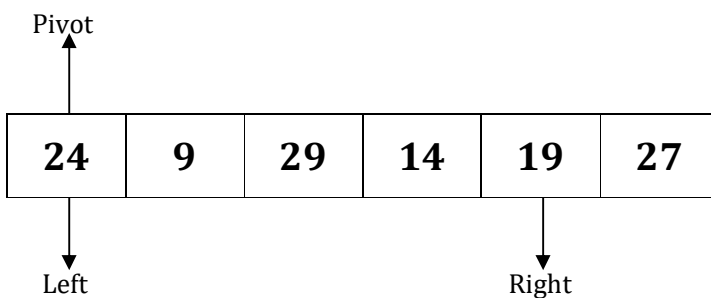
$a[\text{right}] = 27$

$a[\text{pivot}] = 24$

Since, pivot is at left, so algorithm starts from right and move towards left.

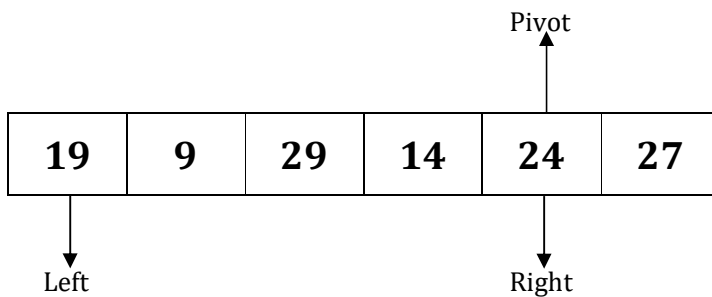


Now, $a[\text{pivot}] < a[\text{right}]$, so algorithm moves forward one position towards left, i.e. –



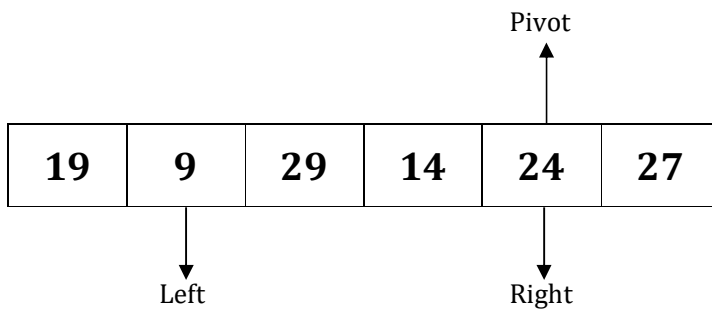
Now, $a[\text{left}] = 24$, $a[\text{right}] = 19$, and $a[\text{pivot}] = 24$.

Because, $a[\text{pivot}] > a[\text{right}]$, so, algorithm will swap $a[\text{pivot}]$ with $a[\text{right}]$, and pivot moves to right, as -

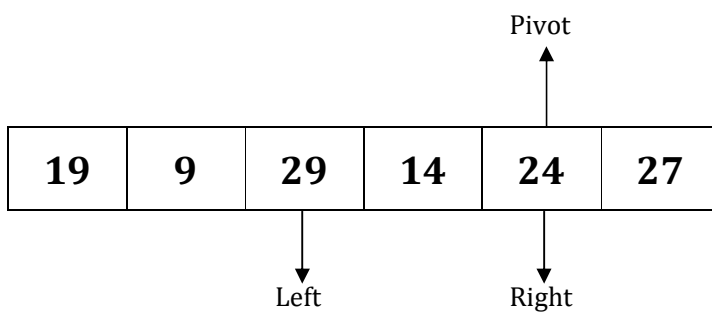


Now, $a[\text{left}] = 19$, $a[\text{right}] = 24$, and $a[\text{pivot}] = 24$. Since, pivot is at right, so algorithm starts from left and moves to right.

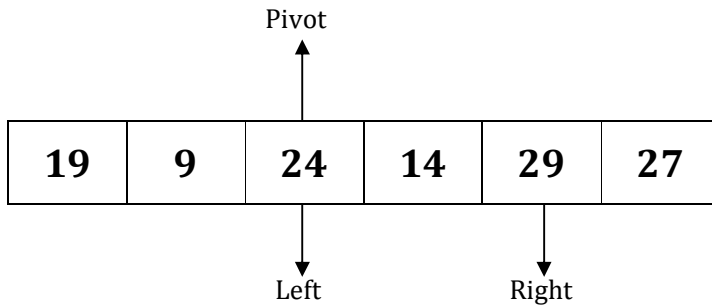
As $a[\text{pivot}] > a[\text{left}]$, so algorithm moves one position to right as -



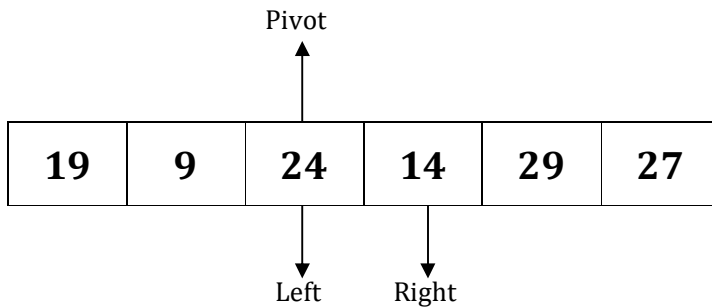
Now, $a[\text{left}] = 9$, $a[\text{right}] = 24$, and $a[\text{pivot}] = 24$. As $a[\text{pivot}] > a[\text{left}]$, so algorithm moves one position to right as -



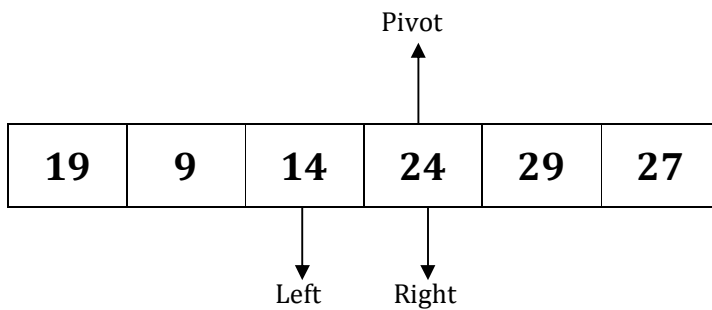
Now, $a[\text{left}] = 29$, $a[\text{right}] = 24$, and $a[\text{pivot}] = 24$. As $a[\text{pivot}] < a[\text{left}]$, so, swap $a[\text{pivot}]$ and $a[\text{left}]$, now pivot is at left, i.e. -



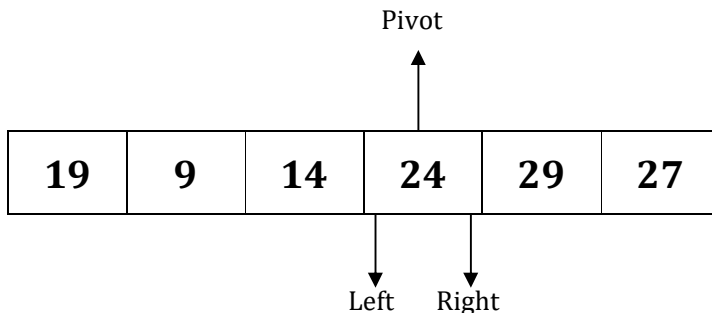
Since, pivot is at left, so algorithm starts from right, and move to left. Now, $a[\text{left}] = 24$, $a[\text{right}] = 29$, and $a[\text{pivot}] = 24$. As $a[\text{pivot}] < a[\text{right}]$, so algorithm moves one position to left, as –



Now, $a[\text{pivot}] = 24$, $a[\text{left}] = 24$, and $a[\text{right}] = 14$. As $a[\text{pivot}] > a[\text{right}]$, so, swap $a[\text{pivot}]$ and $a[\text{right}]$, now pivot is at right, i.e. –



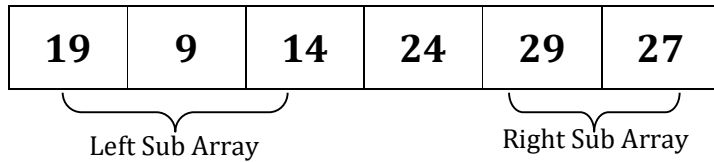
Now, $a[\text{pivot}] = 24$, $a[\text{left}] = 14$, and $a[\text{right}] = 24$. Pivot is at right, so the algorithm starts from left and move to right.



Now, $a[\text{pivot}] = 24$, $a[\text{left}] = 24$, and $a[\text{right}] = 24$. So, pivot, left and right are pointing the same element. It represents the termination of procedure.

Element 24, which is the pivot element is placed at its exact position.

Elements that are right side of element 24 are greater than it, and the elements that are left side of element 24 are smaller than it.

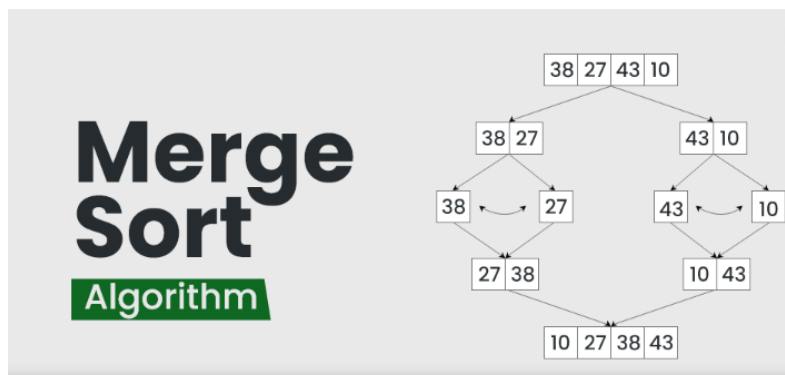


Now, in a similar manner, quick sort algorithm is separately applied to the left and right sub-arrays. After sorting gets done, the array will be –

9	14	19	24	27	29
----------	-----------	-----------	-----------	-----------	-----------

Merge sort is a sorting algorithm that follows the [divide-and-conquer](#) approach. It works by recursively dividing the input array into smaller subarrays and sorting those subarrays then merging them back together to obtain the sorted array.

In simple terms, we can say that the process of **merge sort** is to divide the array into two halves, sort each half, and then merge the sorted halves back together. This process is repeated until the entire array is sorted.



How does Merge Sort work?

Merge sort is a popular sorting algorithm known for its efficiency and stability. It follows the **divide-and-conquer** approach to sort a given array of elements.

Here's a step-by-step explanation of how merge sort works:

1. **Divide:** Divide the list or array recursively into two halves until it can no more be divided.
2. **Conquer:** Each subarray is sorted individually using the merge sort algorithm.
3. **Merge:** The sorted subarrays are merged back together in sorted order. The process continues until all elements from both subarrays have been merged.

Illustration of Merge Sort:

Let's sort the array or list [38, 27, 43, 10] using Merge Sort

Let's look at the working of above example:

Divide:

- [38, 27, 43, 10] is divided into [38, 27] and [43, 10].
- [38, 27] is divided into [38] and [27].
- [43, 10] is divided into [43] and [10].

Conquer:

- [38] is already sorted.
- [27] is already sorted.
- [43] is already sorted.

- **[10]** is already sorted.

Merge:

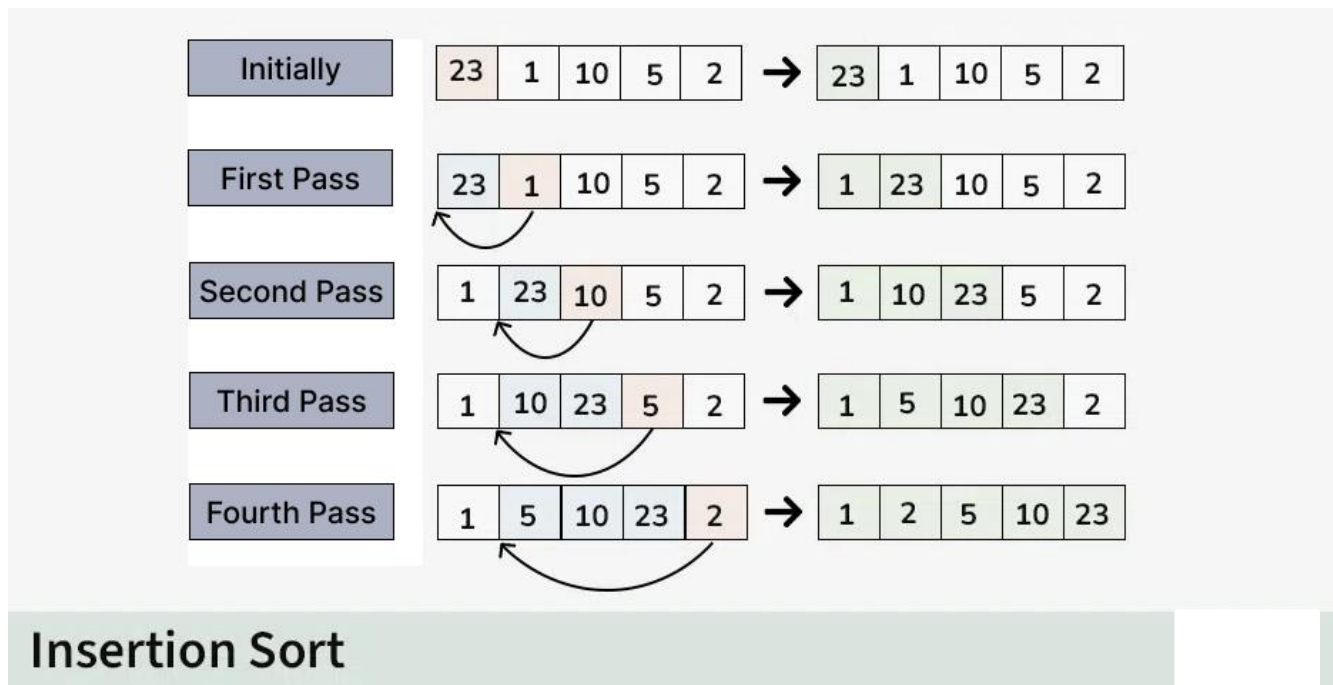
- Merge **[38]** and **[27]** to get **[27, 38]** .
- Merge **[43]** and **[10]** to get **[10,43]** .
- Merge **[27, 38]** and **[10,43]** to get the final sorted list **[10, 27, 38, 43]**

Therefore, the sorted list is **[10, 27, 38, 43]** .

Insertion Sort Algorithm

Insertion sort is a simple sorting algorithm that works by iteratively inserting each element of an unsorted list into its correct position in a sorted portion of the list. It is like sorting playing cards in your hands. You split the cards into two groups: the sorted cards and the unsorted cards. Then, you pick a card from the unsorted group and put it in the right place in the sorted group.

- We start with second element of the array as first element in the array is assumed to be sorted.
- Compare second element with the first element and check if the second element is smaller then swap them.
- Move to the third element and compare it with the first two elements and put at its correct position
- Repeat until the entire array is sorted.



Insertion Sort

Program :

Write a program to sort given Array using Insertion sort technique

```
#include<stdio.h>
```

```
void insert();
```

```
int a[20],n,i,j,temp;
```

```
void main()
```

```
{
```

```
    printf("Enter the limit: ");
```

```
    scanf("%d",&n);
```

```

printf("Enter the %d elements: ",n);

for(i=0; i<n; i++)

    scanf("%d",&a[i]);

printf("Before sorting array elements are - \n");

for(i=0; i<n; i++)

    printf("%d",a[i]);

insert(a, n);

printf("\nAfter sorting array elements are - \n");

for(i=0; i<n; i++)

    printf("%d",a[i]);

getch();
}

void insert()
{
for (i = 1; i < n; i++)
{
    temp = a[i];
    j = i - 1;
    while(j>=0 && temp <= a[j])
    {
        a[j+1] = a[j];
        j = j-1;
    }
    a[j+1] = temp;
}
}

```

Sorting Definitions:

In data structures, sorting refers to the process of arranging elements in a particular order. The most common orders used for sorting are:

- **Ascending order:** where the elements are arranged in increasing order, such as 1, 2, 3, 4, 5.
- **Descending order:** where the elements are arranged in decreasing order, such as 5, 4, 3, 2, 1.

Sorting is an essential operation in many computer programs, as it allows us to organize data in a way that makes it easier to search, analyse, and manipulate. There are several algorithms that can be used to sort data, each with different advantages and disadvantages in terms of time and space complexity. Some common sorting algorithms include:

- a) Bubble Sort
- b) Selection Sort
- c) Insertion Sort
- d) Merge Sort
- e) Quick Sort
- f) Heap Sort
- g) Radix Sort

Each of these algorithms has a different approach to sorting elements, and they can be efficient depending on the size of the data and the desired order.

Bubble Sort

Bubble sort works on the repeatedly swapping of adjacent elements until they are not in the intended order. It is called bubble sort because the movement of array elements is just like the movement of air bubbles in the water. Bubbles in water rise up to the surface; similarly, the array elements in bubble sort move to the end in each iteration.

Although it is simple to use, it is primarily used as an educational tool because the performance of bubble sort is poor in the real world. It is not suitable for large data sets.

Working of Bubble sort Algorithm

Now, let us see the working of Bubble sort Algorithm.

To understand the working of bubble sort algorithm, let us take an unsorted array.

13	32	26	35	10
----	----	----	----	----

First Pass

Sorting will start from the initial two elements. Let compare them to check which is greater.

13	32	26	35	10
----	----	----	----	----

Here, 32 is greater than 13 ($32 > 13$), so it is already sorted. Now, compare 32 with 26.

13	32	26	35	10
----	----	----	----	----

Here, 26 is smaller than 32. So, swapping is required. After swapping new array will look like –

13	26	32	35	10
----	----	----	----	----

Now, compare 32 and 35.

13	26	32	35	10
----	----	----	----	----

Here, 35 is greater than 32. So, there is no swapping required as they are already sorted.

Now, the comparison will be in between 35 and 10.

13	26	32	35	10
----	----	----	----	----

Here, 10 is smaller than 35 that are not sorted. So, swapping is required. Now, we reach at the end of the array. After first pass, the array will be –

13	26	32	10	35
----	----	----	----	----

Second Pass

The same process will be followed for second iteration.

13	26	32	10	35
----	----	----	----	----

13	26	32	10	35
----	----	----	----	----

13	26	32	10	35
----	----	----	----	----

Here, 10 is smaller than 32. So, swapping is required. After swapping, the array will be –

13	26	10	32	35
----	----	----	----	----

13	26	10	32	35
----	----	----	----	----

Third Pass

The same process will be followed for third iteration.

13	26	10	32	35
----	----	----	----	----

13	26	10	32	35
----	----	----	----	----

Here, 10 is smaller than 26. So, swapping is required. After swapping, the array will be –

13	10	26	32	35
----	----	----	----	----

13	10	26	32	35
----	----	----	----	----

13	10	26	32	35
----	----	----	----	----

Fourth pass

Similarly, after the fourth iteration, the array will be –

10	13	26	32	35
----	----	----	----	----

Hence, there is no swapping required, so the array is completely sorted.

Bubble Sort Algorithm

```
begin BubbleSort(arr)
  for all array elements
    if arr[i] > arr[i+1]
      swap(arr[i], arr[i+1])
    end if
  end for
  return arr
end BubbleSort
```

Program

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int a[50],n,i,j,temp=0;

    printf("Enter the limit for the array: ");
```

```
scanf("%d",&n);

printf("Enter the %d elements for the array\n",n);
for(i=0;i<n;i++)
{
    scanf("%d",&a[i]);
}

for(i=0;i<n-1;i++)
{
    for(j=0;j<n-1-i;j++)
    {
        if(a[j]>a[j+1])
        {
            temp=a[j];
            a[j]=a[j+1];
            a[j+1]=temp;
        }
    }
}
for(i=0;i<n;i++)
    printf("%d\n",a[i]);
getch();
}
```

Sorting Definitions:

In data structures, sorting refers to the process of arranging elements in a particular order. The most common orders used for sorting are:

- **Ascending order:** where the elements are arranged in increasing order, such as 1, 2, 3, 4, 5.
- **Descending order:** where the elements are arranged in decreasing order, such as 5, 4, 3, 2, 1.

Sorting is an essential operation in many computer programs, as it allows us to organize data in a way that makes it easier to search, analyse, and manipulate. There are several algorithms that can be used to sort data, each with different advantages and disadvantages in terms of time and space complexity. Some common sorting algorithms include:

- a) Bubble Sort
- b) Selection Sort
- c) Insertion Sort
- d) Merge Sort
- e) Quick Sort
- f) Heap Sort
- g) Radix Sort

(5)

Each of these algorithms has a different approach to sorting elements, and they can be efficient depending on the size of the data and the desired order.

Bubble Sortexchange

Bubble sort works on the repeatedly swapping of adjacent elements until they are not in the intended order. It is called bubble sort because the movement of array elements is just like the movement of air bubbles in the water. Bubbles in water rise up to the surface; similarly, the array elements in bubble sort move to the end in each iteration.

we get sorted array

movement is like bubbles in water

Although it is simple to use, it is primarily used as an educational tool because the performance of bubble sort is poor in the real world. It is not suitable for large data sets.

Working of Bubble sort Algorithm

Now, let us see the working of Bubble sort Algorithm.

To understand the working of bubble sort algorithm, let us take an unsorted array.

13	32	26	35	10
----	----	----	----	----

First Pass

Sorting will start from the initial two elements. Let compare them to check which is greater.

13	32	26	35	10
----	----	----	----	----

Here, 32 is greater than 13 ($32 > 13$), so it is already sorted. Now, compare 32 with 26.

13	32	26	35	10
----	----	----	----	----

Here, 26 is smaller than 32. So, swapping is required. After swapping new array will look like -

13	26	32	35	10
----	----	----	----	----

Now, compare 32 and 35.

13	26	32	35	10
----	----	----	----	----

Here, 35 is greater than 32. So, there is no swapping required as they are already sorted.

Now, the comparison will be in between 35 and 10.

13	26	32	35	10
----	----	----	----	----

Here, 10 is smaller than 35 that are not sorted. So, swapping is required. Now, we reach at the end of the array. After first pass, the array will be -

13	26	32	10	35
----	----	----	----	----

Second Pass

The same process will be followed for second iteration.

13	26	32	10	35
----	----	----	----	----

13	26	32	10	35
----	----	----	----	----

13	26	32	10	35
----	----	----	----	----

Here, 10 is smaller than 32. So, swapping is required. After swapping, the array will be -

13	26	10	32	35
----	----	----	----	----

13	26	10	32	35
----	----	----	----	----

Third Pass

The same process will be followed for third iteration.

13	26	10	32	35
----	----	----	----	----

13	26	10	32	35
----	----	----	----	----

Here, 10 is smaller than 26. So, swapping is required. After swapping, the array will be -

13	10	26	32	35
----	----	----	----	----

13	10	26	32	35
----	----	----	----	----

13	10	26	32	35
----	----	----	----	----

Fourth pass

Similarly, after the fourth iteration, the array will be -

10	13	26	32	35
----	----	----	----	----

Hence, there is no swapping required, so the array is completely sorted.

Bubble Sort Algorithm

```

begin BubbleSort(arr)
  for all array elements
    if arr[i] > arr[i+1]
      swap(arr[i], arr[i+1])
    end if
  end for
  return arr
end BubbleSort

```

Program

```

#include<stdio.h>
#include<conio.h>
void main()
{
  int a[50],n,i,j,temp=0;

  printf("Enter the limit for the array: ");

```



```
scanf("%d",&n);

printf("Enter the %d elements for the array\n",n);
for(i=0;i<n;i++)
{
    scanf("%d",&a[i]);
}

for(i=0;i<n-1;i++)
{
    for(j=0;j<n-1-i;j++)
    {
        if(a[j]>a[j+1])
        {
            temp=a[j];
            a[j]=a[j+1];
            a[j+1]=temp;
        }
    }
}
for(i=0;i<n;i++)
    printf("%d\n",a[i]);
getch();
}
```

Selection Sort

In selection sort, the smallest value among the unsorted elements of the array is selected in every pass and inserted to its appropriate position into the array. It is also the simplest algorithm. It is an in-place comparison sorting algorithm.

In selection sort, the first smallest element is selected from the unsorted array and placed at the first position. After that second smallest element is selected and placed in the second position. The process continues until the array is entirely sorted.

Selection sort is generally used when -

- A small array is to be sorted
- Swapping cost doesn't matter
- It is compulsory to check all elements

Working of Selection sort Algorithm

To understand the working of the Selection sort algorithm, let's take an unsorted array. It will be easier to understand the Selection sort via an example.

12	29	25	8	32	17	40
----	----	----	---	----	----	----

Now, for the first position in the sorted array, the entire array is to be scanned sequentially.

At present, 12 is stored at the first position, after searching the entire array, it is found that 8 is the smallest value.

12	29	25	8	32	17	40
----	----	----	---	----	----	----

So, swap 12 with 8. After the first iteration, 8 will appear at the first position in the sorted array.

8	29	25	12	32	17	40
---	----	----	----	----	----	----

For the second position, where 29 is stored presently, we again sequentially scan the rest of the items of unsorted array. After scanning, we find that 12 is the second lowest element in the array that should be appeared at second position.

8	29	25	12	32	17	40
---	----	----	----	----	----	----

Now, swap 29 with 12. After the second iteration, 12 will appear at the second position in the sorted array. So, after two iterations, the two smallest values are placed at the beginning in a sorted way.

8	12	25	29	32	17	40
---	----	----	----	----	----	----

The same process is applied to the rest of the array elements. Now, we are showing a pictorial representation of the entire sorting process.

8	12	25	29	32	17	40
---	----	----	----	----	----	----

8	12	25	29	32	17	40
---	----	----	----	----	----	----

8	12	17	29	32	25	40
---	----	----	----	----	----	----

8	12	17	29	32	25	40
---	----	----	----	----	----	----

8	12	17	29	32	25	40
---	----	----	----	----	----	----

8	12	17	25	32	29	40
---	----	----	----	----	----	----

8	12	17	25	32	29	40
---	----	----	----	----	----	----

8	12	17	25	32	29	40
---	----	----	----	----	----	----

8	12	17	25	29	32	40
---	----	----	----	----	----	----

Now, the array is completely sorted.

Sorting

→ Sorting: - The process of arranging the elements in a particular order.
 Ascending
 Descending

Merge Sort

How does Merge Sort work?

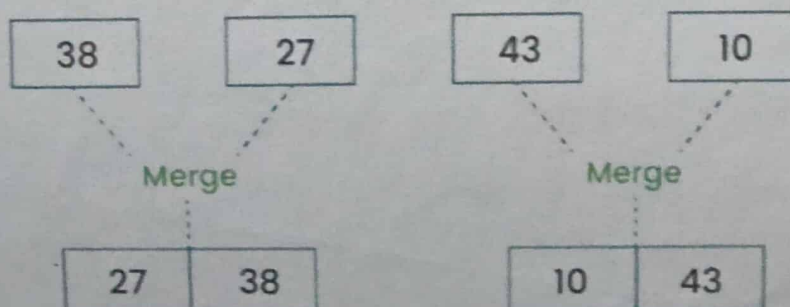
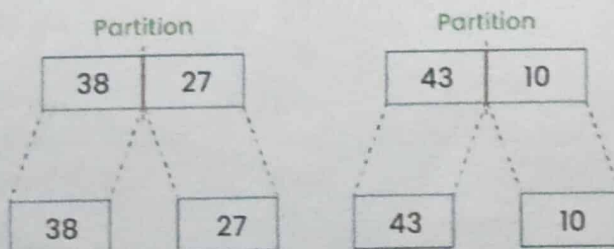
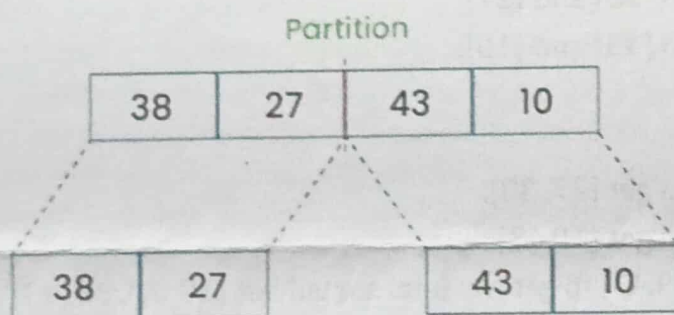
Merge sort is a popular sorting algorithm known for its efficiency and stability. It follows the divide-and-conquer approach to sort a given array of elements.
 div
 merge
 o/n

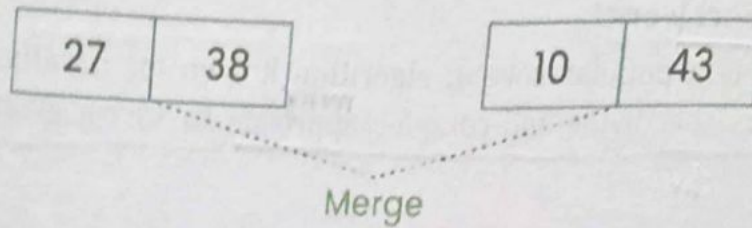
Here's a step-by-step explanation of how merge sort works:

- ① **Divide:** Divide the list or array recursively into two halves until it can no more be divided.
 sort the subarray
- ② **Conquer:** Each subarray is sorted individually using the merge sort algorithm.
- ③ **Merge:** The sorted subarrays are merged back together in sorted order. The process continues until all elements from both subarrays have been merged.

Let's sort the array or list [38, 27, 43, 10] using Merge Sort

follows divide
and merge to
sort given





Let's look at the working of above example:

Divide:

[38, 27, 43, 10] is divided into [38, 27] and [43, 10].

[38, 27] is divided into [38] and [27].

[43, 10] is divided into [43] and [10].

Conquer & Merge:

Merge [38] and [27] to get [27, 38].

Merge [43] and [10] to get [10, 43].

Merge [27, 38] and [10, 43] to get the final sorted list [10, 27, 38, 43]

Therefore, the sorted list is [10, 27, 38, 43].

Quick Sort

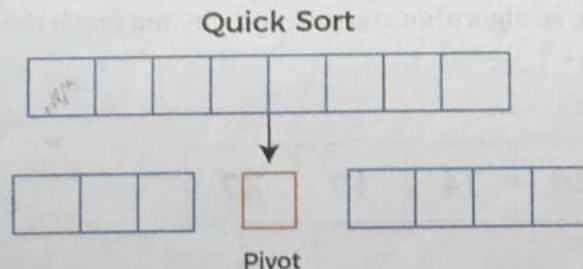
Quick sort is the widely used sorting algorithm that makes comparisons in average case for sorting an array of n elements. It is a faster and highly efficient sorting algorithm. This algorithm follows the divide and conquers approach.

Divide and conquer is a technique of breaking down the algorithms into sub problems, then solving the sub problems, and combining the results back together to solve the original problem.

- **Divide:** In Divide, first pick a pivot element. After that, partition or rearrange the array into two sub-arrays such that each element in the left sub-array is less than or equal to the pivot element and each element in the right sub-array is larger than the pivot element.
- **Conquer:** Recursively, sort two subarrays with Quick sort.
- **Combine:** Combine the already sorted array.

Quick sort picks an element as pivot, and then it partitions the given array around the picked pivot element. In quick sort, a large array is divided into two arrays in which one holds values that are smaller than the specified value (Pivot), and another array holds the values that are greater than the pivot.

After that, left and right sub-arrays are also partitioned using the same approach. It will continue until the single element remains in the sub-array.



Choosing the pivot:

Some of the ways of choosing a pivot are as follows -

- Pivot can be random, i.e. select the random pivot from the given array.
- Pivot can either be the rightmost element or the leftmost element of the given array.
- Select median as the pivot element.

Quick sort → sorting of list based on pivot element, usually 1st element or p.
(Arrange) elements by selecting Pivot and dividing the array into two parts — one with elements smaller than the pivot & one with elements larger.
This process is repeated for each part until the entire array is sorted.
Among all sorting techniques it is fastest & best.

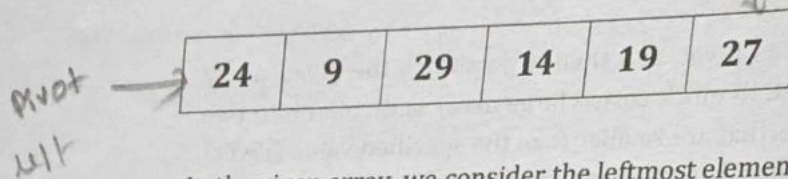
Algorithm:

QUICKSORT (array A, start, end)

```
{  
    if (start < end)  
    {  
        p = partition(A, start, end)  
        QUICKSORT (A, start, p - 1)  
        QUICKSORT (A, p + 1, end)  
    }  
}
```

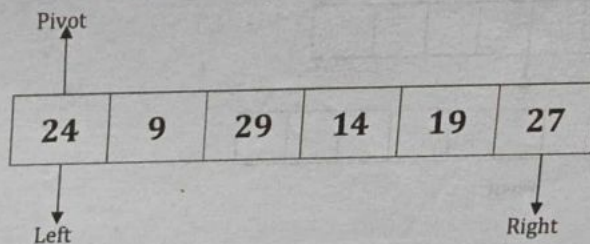
Working of Quick Sort Algorithm

To understand the working of quick sort, let's take an unsorted array. It will make the concept more clear and understandable.

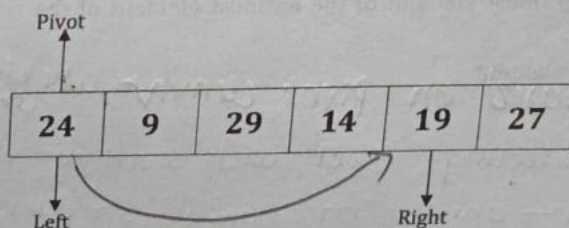


In the given array, we consider the leftmost element as pivot. So, in this case,
 $a[\text{left}] = 24$
 $a[\text{right}] = 27$
 $a[\text{pivot}] = 24$

Since, pivot is at left, so algorithm starts from right and move towards left.



Now, $a[\text{pivot}] < a[\text{right}]$, so algorithm moves forward one position towards left,
i.e. - $24 < 27$ ✓ *pivot smaller, right side move*



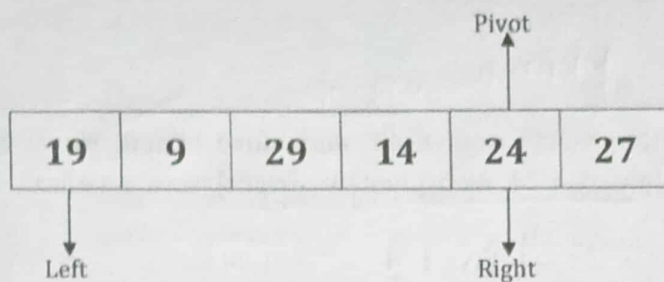
$a[\text{pivot}] > a[\text{right}]$
 $24 > 19$

pivot larger, swap

Now, $a[\text{left}] = 24$, $a[\text{right}] = 19$, and $a[\text{pivot}] = 24$.

Because, $a[\text{pivot}] > a[\text{right}]$, so, algorithm will swap $a[\text{pivot}]$ with $a[\text{right}]$, and pivot moves to right, as -

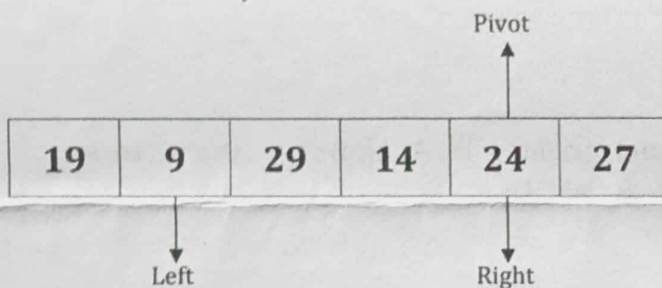
$a[\text{pivot}] > a[\text{right}]$
24 > 19.



Now, $a[\text{left}] = 19$, $a[\text{right}] = 24$, and $a[\text{pivot}] = 24$. Since, pivot is at right, so algorithm starts from left and moves to right.

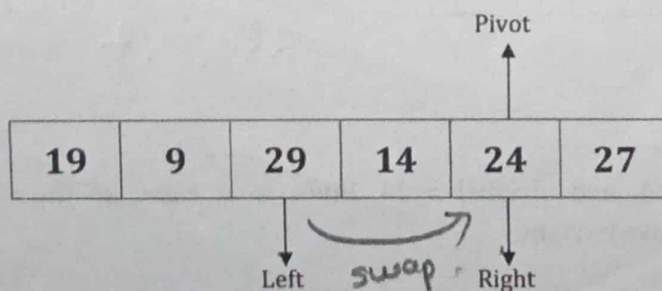
As $a[\text{pivot}] > a[\text{left}]$, so algorithm moves one position to right as -

24 > 19 move one position



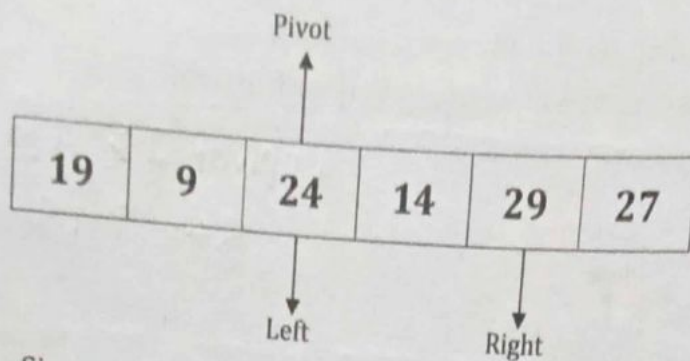
Now, $a[\text{left}] = 9$, $a[\text{right}] = 24$, and $a[\text{pivot}] = 24$. As $a[\text{pivot}] > a[\text{left}]$, so algorithm moves one position to right as -

24 > 9 ✓ one position



Now, $a[\text{left}] = 29$, $a[\text{right}] = 24$, and $a[\text{pivot}] = 24$. As $a[\text{pivot}] < a[\text{left}]$, so, swap $a[\text{pivot}]$ and $a[\text{left}]$, now pivot is at left, i.e. -

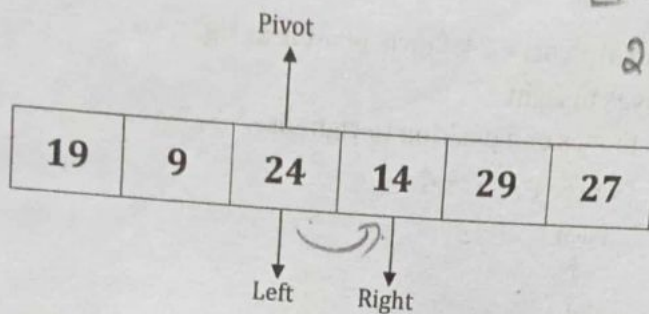
$a[\text{pivot}] < a[\text{left}]$
24 < 29 swap.



Since, pivot is at left, so algorithm starts from right, and move to left. Now, $a[\text{left}] = 24$, $a[\text{right}] = 29$, and $a[\text{pivot}] = 24$. As $a[\text{pivot}] < a[\text{right}]$, so algorithm moves one position to left, as -

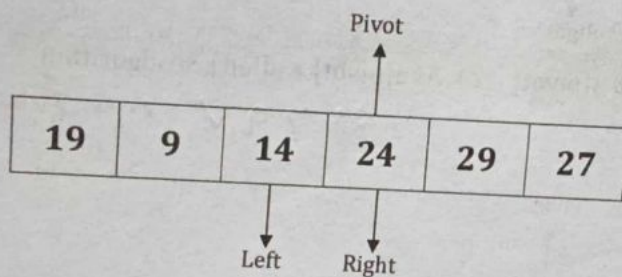
$$a[\text{pivot}] < a[\text{right}]$$

$$24 < 29$$



Now, $a[\text{pivot}] = 24$, $a[\text{left}] = 24$, and $a[\text{right}] = 14$. As $a[\text{pivot}] > a[\text{right}]$, so, swap $a[\text{pivot}]$ and $a[\text{right}]$, now pivot is at right, i.e. -

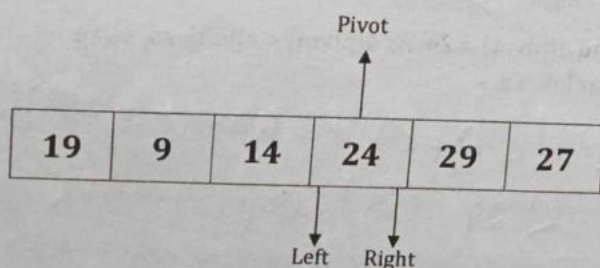
$$24 > 14 \quad \text{Swap}$$



$$a[\text{pivot}] > a[\text{right}]$$

$$24 > 14$$

Now, $a[\text{pivot}] = 24$, $a[\text{left}] = 14$, and $a[\text{right}] = 24$. Pivot is at right, so the algorithm starts from left and move to right.



Now, $a[\text{pivot}] = 24$, $a[\text{left}] = 24$, and $a[\text{right}] = 24$. So, pivot, left and right are pointing the same element. It represents the termination of procedure.

Element 24, which is the pivot element is placed at its exact position.

Elements that are right side of element 24 are greater than it, and the elements that are left side of element 24 are smaller than it.

19	9	14	24	29	27
----	---	----	----	----	----

Left Sub Array

smaller than 24

Right Sub Array

greater than 24

Now, in a similar manner, quick sort algorithm is separately applied to the left and right sub-arrays. After sorting gets done, the array will be -

9	14	19	24	27	29
---	----	----	----	----	----

sorted array

Importance of Sorting

* easier to search, analyse, manipulate,

sorting types $\Rightarrow$ it has advantage/disadvantage based on time, space complexity

- ① Merge sort
- ② Quick sort
- ③ Bubble sort
- ④ Insertion sort
- ⑤ Selection sort